

PROGMENTURES

SOLUTIONS PVT. LTD.

PROJECT TECHNICAL MANUAL & SYSTEM PLAN

PROGMENTURES PRS PORTAL

Complete Documentation for Recruitment, Registration & Internship System Portal. Detailed workflows, modules, database structures, active .env parameters, credentials directory, and deployment instructions.

PREPARED FOR

Progmentures Solution Pvt. Ltd.

DOCUMENT VERSION

1.0.0 (Production)

Table of Contents / Index

1. Executive Overview & Company Context	3
2. Technology Stack & Platform Architecture	3
3. Environment Configuration Parameters (From .env)	4
4. System Authentication & Credentials Directory	4
5. Detailed User Roles & Functionalities	5
6. Core Architecture Workflows	6
7. Comprehensive Database Model Catalog (18 Models)	7
8. Backend API Endpoint Reference	11
9. Installation & Deployment Guide	12
10. Operations & Troubleshooting Directory	13

■ 1. Executive Overview & Company Context

Progmentures Solution Pvt. Ltd. is an enterprise focused on student development, internships, and skill certification. The **Progmentures PRS Portal** (Recruitment, Registration & Internship System) provides a unified tool that manages student onboarding, task coordination, performance tracking, representativePayout referrals, payments, and certificates.

By implementing a unified multi-portal interface and clean MongoDB schemas, the PRS platform ensures data integrity and seamless coordination between system administrators, trainer/employees, candidate/aspirants, and referral partners.

■ 2. Technology Stack & Platform Architecture

The architecture represents a standard MERN stack configuration, modified to support secure uploads and localized document checks:

Tier	Technology	Implementation Context
Frontend UI	React (v18.2.0)	Single Page Application rendering dashboards dynamically based on logged-in user tokens.
Build/Dev Bundle	Vite (v5.0.8)	Vite bundler providing highly optimized build assets and HMR during development.
Routing Engine	React Router DOM (v6.20.0)	Client-side route guards rendering page views and handling unauthorized redirects.
Backend Framework	Express.js (v4.18.2) on Node.js	Processes requests, validates request bodies, filters data via models, and streams uploads.
Database Layers	MongoDB & Mongoose (v8.0.0)	Structured schema definitions with populate lookups for students, trainer evaluations, and representative payouts.
Session Security	jsonwebtoken (JWT) (v9.0.2)	Token encryption with 24-hour expiration verifyToken middleware logic.
Hashing Security	bcryptjs (v2.4.3)	Password salting and validation logic.
Upload Middleware	Multer (v2.0.2)	Dynamic disk engines configured to handle files across segmented folder locations.

3. Environment Configuration Parameters (From .env)

Below is the active environment file configuration extracted from the `backend/.env` configuration:

```
# Server Configuration
PORT=5000
NODE_ENV=production
PASSWORD_SALT_ROUNDS=10

# MongoDB Connection String (Production Database Cluster)
MONGO_URI=mongodb+srv://progrenturesolution_db_user:uzYijE9JQgo39HL@cluster0progrentures

# JSON Web Token Private Encryption Key
JWT_SECRET=5ae7facd5dc579c740b1573cb68995e12d9f7e64a0b25456c4f073b39b08eddc3543c3acc45605c
```

 Production Security Notice

Ensure that the `JWT_SECRET` environment variable is updated to a unique, randomized 256-bit string in staging environments. The database connection utilizes a cloud MongoDB Atlas instance configured for connection pooling.

4. System Authentication & Credentials Directory

During the backend application startup phase (`backend/config/db.js`), Mongoose seeds default system administrator credentials into the database. If an administrator record is missing, it is created. If present, credentials are automatically synchronized:

System Admin Account	Initial Password	Assigned Role
admin@progrentures.com	PRSPortal@2026	Global Master Administrator
aniruddharaut2004@gmail.com	Raut@2004	System Developer Administrator
rohanguhatol4@gamil.com	Rohan@2004	System Developer Administrator

****Authentication Process**:** Users authenticate via the unified login page tab selectors. The backend checks credentials, signs a JWT token, and returns user data. The token, user context,

and user role are stored in the client's `localStorage` .

5. Detailed User Roles & Functionalities

Admin Dashboard

Target Endpoint: /admin-dashboard

- **Aspirant CRUD:** Onboard students, assign groups, and update designations.
- **Recycle Bin (Soft-Delete):** Safeguard to store deleted aspirants. Admin can view, restore, or delete permanently.
- **Employee/Trainer Management:** Create trainer profiles and allocate student groups.
- **PIGR Representative Onboarding:** Manage representative scanners, agreements, and details.
- **Payment Logs:** Create and update payments records for students.
- **Certificate Assignment:** Assign PDF certificates with auto-deletion properties.
- **Job board:** CRUD operations on jobs and internships with reopen actions.
- **Broadcast Engine:** Send announcements with PDF attachments to all users.

Aspirant Dashboard (Intern / Student)

Target Endpoint: /intern-dashboard

- **Task Center:** View assigned tasks, download attachment briefs, and monitor progress bars.
- **Submission Panel:** Update tasks progress (25%, 50%, 75%, 100% - Submit). Dropdown locks at 100%.
- **Direct Chat:** Post questions inside tasks, read responses from admins.
- **Evaluations Log:** Real-time access to scores submitted by trainers (Aptitude, GD, Assessments, and Training).
- **Certificates Hub:** Access and download active certificates assigned to them.
- **Notifications Board:** Read announcements and retrieve attachments.
- **Job Openings:** Browse placement preparation jobs.

Employee Dashboard (Trainer)

Target Endpoint: `/trainer-dashboard`

- **Student Rosters:** Monitor assigned student lists, contact numbers, and joining dates.
- **Evaluation Matrix:** Input and edit interview ratings, aptitude cards, and training metrics.
- **Status Controls:** Alter pipeline status of students (e.g. active, terminated).
- **Event Scheduler:** Create and manage scheduled group discussion (GD) classes and mock interviews.
- **Work Assignments:** Read assignments issued by Admin.

PIGR Dashboard (Representative)

Target Endpoint: `/representative-dashboard`

- **Direct Registrations:** Create student accounts under their PIGR referral code.
- **Onboarding Uploads:** Submit enrollment forms, agreement letters, and fee receipts.
- **Earnings Monitor:** View paid and pending payout receipts from administrators.
- **Referred Stats:** Track active referred student analytics.

6. Core Architecture Workflows

6.1 Task Progress & Verification Pipeline

The platform enforces a strict task progression state machine to ensure aspirants submit their work for validation. When an administrator creates a task, they specify the details, deadline, and can upload a PDF brief.

The aspirant accesses the task and updates the progress. The status transitions as follows: `Assigned (0%)` → `In Progress (25%, 50%, 75%)` → `Pending Approval (100%)` → `Completed (Approved by Admin)`. Once set to 100%, the selection menu is locked to prevent modification. The administrator reviews the work, exchanges comments with the aspirant in the thread, and clicks "Approve", which updates the status to `Completed` and updates the aspirant's progress charts.

6.2 Automated Certificate Expiration Policy

To optimize storage resources on the production server, assigned certificates are saved with an expiration timeline. When the administrator assigns certificates, they are stored with an `expiresAt` date set to exactly 5 days (120 hours) from creation.

The backend incorporates an automatic cleanup engine (`certificateController.cleanupExpiredCertificates`):

- Runs automatically on server startup.
- Runs hourly via standard `setInterval` loops.
- Locates all records where `expiresAt` is less than or equal to the current date, unlinks the target PDF from the server storage, and deletes the record from the database.

6.3 PIGR Representative Student Approval Onboarding

Referral representatives upload documentation files when adding a student. The backend stores these documents in a pending status under the student model. The student cannot log in until verified.

The administrator accesses the "Pending Approvals" dashboard, reviews the receipts and letters, and approves the entry. The system changes the account status to active and sends the credentials mail via Nodemailer.

7. Comprehensive Database Model Catalog (18 Models)

The database contains 18 models stored in `backend/models/` :

7.1 Admin Schema (Admin.js)

```
{
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true },
  role: { type: String, default: 'admin' }
}
```

7.2 Intern / Student Schema (Intern.js)

```
{
  studentType: { type: String, required: true, enum: ['Internship', 'SMS Program'] },
  internId: { type: String, required: true, unique: true }, // Format: PIID/ or PSMS/
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  plainPassword: { type: String }, // For credential reminders
  mobile: { type: String, required: true },
  role: { type: String, default: 'intern' },
  status: { type: String, enum: ['Active', 'Terminated', 'Placed'], default: 'Active' },
  isDeleted: { type: Boolean, default: false }, // Soft delete support
  // Internship Fields
  domain: { type: String },
  stipendType: { type: String, enum: ['Stipend', 'Unstipend'], default: 'Unstipend' },
  stipendAmount: { type: String },
  joiningDate: { type: Date },
  endingDate: { type: Date },
  duration: { type: String },
  collegeName: { type: String },
  branch: { type: String },
  yearOfStudy: { type: String },
  // SMS Program Fields
  paymentDoneBy: { type: String },
  dateOfPayment: { type: Date },
  transactionId: { type: String },
  paymentAmount: { type: String },
  completedFees: { type: String },
  pendingFees: { type: String },
  lastPaymentDate: { type: Date },
  currentDesignation: { type: String },
  suggestedDomain: { type: String },
  currentQualification: { type: String },
  instituteName: { type: String },
  instituteLocation: { type: String },
  enrolmentDate: { type: Date },
  enrolBatchMonth: { type: String },
  totalFees: { type: String },
  firstPaymentAmount: { type: String },
  firstPaymentDate: { type: Date },
  secondPaymentAmount: { type: String },
  secondPaymentDate: { type: Date },
  finalPaymentAmount: { type: String },
  finalPaymentDate: { type: Date },
  // Relationships
  assignedTrainer: { type: mongoose.Schema.Types.ObjectId, ref: 'Trainer' },
  assignedGroups: [{ type: mongoose.Schema.Types.ObjectId, ref: 'StudentGroup' }],
  documents: {
    welcomeLetter: { filename: String, filepath: String, uploadedAt: Date },
    smsProgramEnrollmentLetter: { filename: String, filepath: String, uploadedAt: Date },
  }
}
```

```
offerLetter: { filename: String, filepath: String, uploadedAt: Date },  
paymentReceipt: { filename: String, filepath: String, uploadedAt: Date }  
}  
}
```

7.3 Trainer Schema (Trainer.js)

```
{
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  mobile: { type: String },
  specialization: [String],
  role: { type: String, default: 'trainer' },
  assignedStudents: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Intern' }],
  assignedGroups: [{ type: mongoose.Schema.Types.ObjectId, ref: 'StudentGroup' }],
  workAssignments: [{
    title: String,
    description: String,
    assignedDate: { type: Date, default: Date.now },
    dueDate: Date,
    assignedStudents: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Intern' }]
  }]
}
```

7.4 Representative / PIGR Schema (Representative.js)

```
{
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  mobile: { type: String },
  role: { type: String, default: 'representative' },
  upiScanner: { type: String },
  pgirSelectionLetter: { type: String },
  internshipOfferLetter: { type: String }
}
```

7.5 Task Schema (Task.js)

```
{
  title: { type: String, required: true },
  description: { type: String, required: true },
  deadline: { type: Date, required: true },
  assignedTo: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern', required: true },
  taskDocument: { type: String }, // Path to task brief PDF
  progress: { type: Number, default: 0 },
  status: { type: String, enum: ['Assigned', 'In Progress', 'Pending Approval', 'Completed'], required: true },
  feedback: { type: String },
  feedbackRead: { type: Boolean, default: false },
  messages: [{
    sender: { type: String, required: true },
    senderRole: { type: String, enum: ['admin', 'intern'], required: true },
    text: { type: String, required: true },
    timestamp: { type: Date, default: Date.now }
  }]
}
```

7.6 Certificate Schema (Certificate.js)

```
{
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern', required: true },
  name: { type: String, required: true },
  filename: { type: String, required: true },
  filepath: { type: String, required: true },
  expiresAt: { type: Date, required: true } // Auto clean trigger: Date.now + 5 days
}
```

7.7 Interview Performance Schema (Interview.js)

```
{
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern', required: true },
  trainerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Trainer', required: true },
  interviewDate: { type: Date, default: Date.now },
  roundType: { type: String, enum: ['Group Discussion (GD)', 'Mock Interview', 'Technical'],
  confidence: { type: Number, min: 1, max: 10 },
  communication: { type: Number, min: 1, max: 10 },
  technicalSkills: { type: Number, min: 1, max: 10 },
  behavioral: { type: Number, min: 1, max: 10 },
  overallScore: { type: Number },
  feedback: { type: String }
}
```

7.8 Aptitude Test Schema (Aptitude.js)

```
{
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern', required: true },
  trainerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Trainer', required: true },
  testDate: { type: Date, default: Date.now },
  quantitative: { type: Number, min: 0, max: 100 },
  logical: { type: Number, min: 0, max: 100 },
  verbal: { type: Number, min: 0, max: 100 },
  overallScore: { type: Number },
  feedback: { type: String }
}
```

7.9 Technology Assessment Schema (Assessment.js)

```
{
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern', required: true },
  trainerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Trainer', required: true },
  assessmentDate: { type: Date, default: Date.now },
  subjectName: { type: String, required: true }, // e.g. Node.js, React, Data structures
  marksObtained: { type: Number, min: 0, max: 100 },
  totalMarks: { type: Number, default: 100 },
  feedback: { type: String }
}
```

7.10 Training Performance Schema (Training.js)

```
{
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern', required: true },
  trainerId: { type: mongoose.Schema.Types.ObjectId, ref: 'Trainer', required: true },
  recordedDate: { type: Date, default: Date.now },
  attendance: { type: Number, min: 0, max: 100 }, // Attendance percentage
  participation: { type: Number, min: 1, max: 10 },
  understanding: { type: Number, min: 1, max: 10 },
  overallProgress: { type: Number }, // Computed score
  feedback: { type: String }
}
```

7.11 Activity Log Schema (Activity.js)

```
{
  activityType: { type: String, required: true }, // e.g., 'Student Status Change'
  description: { type: String, required: true },
  performedBy: { type: String, required: true }, // Admin Email/Trainer Name
  targetStudent: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern' },
  timestamp: { type: Date, default: Date.now }
}
```

7.12 Notification Schema (Notification.js)

```
{
  title: { type: String, required: true },
  message: { type: String, required: true },
  targetAudience: { type: String, enum: ['All', 'intern', 'trainer', 'representative'], default: 'All' },
  attachment: { type: String }, // Path to attachment file
  attachmentName: { type: String },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'Admin' },
  createdAt: { type: Date, default: Date.now }
}
```


7.13 Admin Student Payment Schema (AdminPayment.js)

```
{
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Intern', required: true },
  amount: { type: Number, required: true },
  paymentDate: { type: Date, required: true },
  paymentType: { type: String, enum: ['First Installment', 'Second Installment', 'Final Payment'], required: true },
  transactionId: { type: String },
  remarks: { type: String }
}
```

7.14 Representative Notification Schema (RepresentativeNotification.js)

```
{
  representativeId: { type: mongoose.Schema.Types.ObjectId, ref: 'Representative', required: true },
  title: { type: String, required: true },
  message: { type: String, required: true },
  isRead: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
}
```

7.15 Representative Payout Schema (RepresentativePayout.js)

```
{
  representativeId: { type: mongoose.Schema.Types.ObjectId, ref: 'Representative', required: true },
  amount: { type: Number, required: true },
  payoutDate: { type: Date, required: true },
  paymentMethod: { type: String, required: true },
  transactionId: { type: String },
  status: { type: String, enum: ['Pending', 'Completed'], default: 'Pending' },
  remarks: { type: String }
}
```

7.16 Student Group Schema (StudentGroup.js)

```
{
  name: { type: String, required: true, unique: true },
  description: { type: String },
  students: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Intern' }],
  assignedTrainer: { type: mongoose.Schema.Types.ObjectId, ref: 'Trainer' },
  createdAt: { type: Date, default: Date.now }
}
```

7.17 RepStudent Schema (RepStudent.js)

```
{
  representativeId: { type: mongoose.Schema.Types.ObjectId, ref: 'Representative', required: true },
  name: { type: String, required: true },
  email: { type: String, required: true },
  mobile: { type: String, required: true },
  suggestedDomain: { type: String, required: true },
  status: { type: String, enum: ['Pending', 'Approved', 'Rejected'], default: 'Pending' },
  smsProgramEnrollmentLetter: { type: String },
  offerLetter: { type: String },
  paymentReceipt: { type: String },
  createdAt: { type: Date, default: Date.now }
}
```

7.18 Job Posting Schema (JobPosting.js)

```
{
  title: { type: String, required: true },
  companyName: { type: String, required: true },
  location: { type: String, required: true },
  jobType: { type: String, enum: ['Full Time', 'Internship', 'Contract'], required: true },
  experienceRequired: { type: String },
  salaryRange: { type: String },
  description: { type: String, required: true },
  requirements: [String],
  status: { type: String, enum: ['Open', 'Closed'], default: 'Open' },
  postedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'Admin' },
  createdAt: { type: Date, default: Date.now }
}
```

8. Backend API Endpoint Reference

The backend is structured into REST modules mapped in `backend/routes/` :

8.1 Authentication Routes (authRoutes.js)

- `POST /api/auth/admin-login` - Public. Logs in admin, returns JWT.
- `POST /api/auth/intern-login` - Public. Logs in intern using `internId` , returns JWT.

8.2 Admin Functions (adminRoutes.js)

- `POST /api/admin/add-intern` - Admin. Onboards a student (supports file uploads: `welcomeLetter`, `offerLetter`).
- `GET /api/admin/interns` - Admin. Lists all non-deleted students.
- `DELETE /api/admin/intern/:id` - Admin. Soft-deletes student (moves to recycle bin).
- `GET /api/admin/deleted-interns` - Admin. Views recycle bin.
- `PATCH /api/admin/intern/:id/restore` - Admin. Restores soft-deleted student.
- `DELETE /api/admin/intern/:id/permanent` - Admin. Permanently deletes student.
- `POST /api/admin/add-trainer` - Admin. Registers trainer.
- `GET /api/admin/trainers` - Admin. Lists all trainers.
- `POST /api/admin/assign-students` - Admin. Allocates student IDs to trainer.
- `POST /api/admin/add-representative` - Admin. Registers referral representative.
- `POST /api/admin/groups` - Admin. Creates a student group.
- `POST /api/admin/payments` - Admin. Logs fees payment transaction.

8.3 Task Functions (taskRoutes.js)

- `POST /api/task/admin/create-task` - Admin. Assigns task, uploads taskDocument.
- `PUT /api/task/admin/approve-task/:taskId` - Admin. Approves submitted task (status becomes Completed).
- `GET /api/task/intern/tasks` - Intern. Fetches assigned tasks.
- `PUT /api/task/intern/update-task/:taskId` - Intern. Updates task completion percentage.
- `POST /api/task/intern/team-message/:taskId` - Intern. Comments on task.
- `GET /api/task/intern/my-records` - Intern. Retrieves evaluations (GD, technical interview, training progress, exams).

- `GET /api/task/intern/my-certificates` - Intern. Retrieves active certificates.

8.4 Trainer Functions (trainerRoutes.js)

- `POST /api/trainer/login` - Public. Logs in trainer.
- `GET /api/trainer/students` - Trainer. View assigned students.
- `POST /api/trainer/interviews` - Trainer. Log GD / Interview scorecard.
- `POST /api/trainer/aptitude` - Trainer. Log Aptitude scores.
- `POST /api/trainer/assessments` - Trainer. Log exam scorecards.

8.5 Representative Functions (representativeRoutes.js)

- `POST /api/representative/login` - Public. Representative login.
- `POST /api/representative/students` - Representative. Registers student under referral code, uploads receipts.
- `GET /api/representative/payouts` - Representative. View payouts.

9. Installation & Deployment Guide

Follow these instructions to install, configure, and launch the portal on your workspace environment.

9.1 Automated Windows Execution Setup

Double-click the `setup.bat` script or run the following command in PowerShell:

```
# Execution policy bypass if script is locked
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.setup.ps1
```

This script checks prerequisites, runs `npm install` on both front-end and back-end workspaces, and handles initial directory creations.

9.2 Manual Step-by-Step Installation

1. **Database Verification:** Ensure local MongoDB is active or configure access to your MongoDB Atlas cluster: `mongod --dbpath="C:\data\db"`

2. **Install Backend Dependencies:**

```
cd backend
npm install
```

3. **Configure Env Variables:** Create `backend/.env` with parameters specified in Section 3.

4. **Start API Server:**

```
npm run dev
```

The backend starts on `http://localhost:5000`. Default admin credentials are automatically synchronized during the database connection phase.

5. **Install Frontend Dependencies:**

```
cd ../frontend  
npm install
```

6. Launch Client Application:

```
npm run dev
```

The Vite server runs the client site at `http://localhost:3000` .

10. Operations & Troubleshooting Directory

10.1 MongoDB Connection Terminated (Refused)

- **Diagnosis:** The server console displays `MongooseServerSelectionError` .
- **Remedy:** Verify if your local MongoDB service is active. Open Command Prompt and execute `net start MongoDB` . If you are referencing a remote cloud database, check the Mongo connection string parameters in the `.env` file.

10.2 Multer Validation Exceptions

- **Diagnosis:** The API logs print validation errors, or uploads fail.
- **Remedy:** The tasks and certificates modules require files to be in PDF format. Verify that the file extensions are valid. Ensure the directory paths (e.g. `backend/uploads/students`) exist and have write permissions enabled.

10.3 CORS Access Violations

- **Diagnosis:** Browser consoles display CORS headers block warnings.
- **Remedy:** The backend uses the CORS package to whitelist ports. Ensure that the backend configuration allows requests from `http://localhost:3000` or your deployment domain. Verify that `frontend/src/services/api.js` is configured with the correct `API_BASE_URL` .

10.4 Auto-Expiry Cleanups Failure

- **Diagnosis:** Stale certificates continue to reside on disk beyond the 5-day expiration limit.
- **Remedy:** Check that the backend server is running continuously. The certificate cleanup engine depends on the server's event loop. Restarting the backend server triggers an automatic cleanup cycle on boot.